


# Librerías JavaScript aplicadas al proyecto BackToMe

Kimberly Alexandra Rivera López (RL261989)

Andrea Michelle Jiménez Aguilar (JA262091)

Universidad Don Bosco

DAW901 G01T (Virtual): Desarrollo de Aplicaciones Web con Software Interpretados en el Cliente

Ciclo II – 2026

Alexander Alberto Siguenza Campos

9 de noviembre de 2026

# Indice

Resumen ..... 3

Introducción ..... 3

Método y alcance ..... 4

Componentes y reutilización ..... 4

Estado y eventos ..... 4

Enrutamiento y herramientas ..... 5

Aplicación de React en BackToMe ..... 5

Aportes de las integrantes ..... 6

API externa de Cloudinary ..... 6

Análisis técnico y seguridad ..... 7

Conclusiones ..... 8

Referencias ..... 8

## Resumen

Este informe documenta la integración de React en BackToMe, una aplicación web de objetos perdidos y encontrados de la Universidad Don Bosco, y el consumo de la API externa de Cloudinary para guardar fotografías de publicaciones. Se revisaron los archivos fuente y se contrastaron las decisiones técnicas con documentación oficial. Andrea preparó las pantallas React de inicio de sesión y registro, con rutas, estado de formulario y componentes reutilizables. El aporte individual de Kimberly fue conectar Cloudinary al flujo de publicación: el servidor valida y firma la solicitud, envía la imagen al servicio y guarda en MySQL la URL segura recibida. Se explica el papel de componentes, estado, enrutamiento y herramientas de compilación; además, se describe el flujo navegador-servidor-Cloudinary-base de datos y los controles para proteger credenciales. El alcance se delimita con precisión: las pantallas de acceso están migradas a React, mientras el dashboard y los módulos de objetos, solicitudes y mensajes conservan HTML, CSS y JavaScript. Se concluye que React mejora la organización y reutilización de la interfaz de acceso, y que Cloudinary incorpora almacenamiento externo directamente al flujo de reportes.


**Palabras clave:** React, componentes, estado, Cloudinary, API externa, BackToMe

**Integración de React y Cloudinary en BackToMe**

## Introducción

BackToMe permite a la comunidad universitaria publicar objetos perdidos o encontrados, consultar reportes y coordinar solicitudes. En esta entrega el equipo incorporó React en las pantallas de acceso y conectó una API externa al flujo de publicaciones. Este informe explica el framework elegido, documenta su aplicación y distingue el aporte de cada integrante.

React se presenta oficialmente como una biblioteca de JavaScript para construir interfaces mediante componentes. La consigna académica lo clasifica entre los frameworks permitidos; por eso, se emplea el término framework al referirse al requisito y biblioteca al

describir la herramienta con precisión. React compone y actualiza la interfaz; Vite proporciona el servidor de desarrollo y la compilación, y React Router organiza las rutas (React, s. f.-a, s. f.-b; React Router, s. f.; Vite, s. f.).

## Método y alcance

Se revisaron package.json, react-app/src, react-app/public, server.js, js/app.js, .env.example y las migraciones SQL. Se identificaron las versiones declaradas por el proyecto y se contrastaron componentes, estado, rutas y cargas firmadas con documentación oficial de React, React Router, Vite y Cloudinary. El informe describe el código de esta entrega; no afirma que todos los módulos se hayan migrado ni que la aplicación haya sido evaluada en un entorno de producción.

## React como framework de interfaz

### Componentes y reutilización

Los componentes son unidades de interfaz que pueden combinarse para construir páginas. JSX permite describir la estructura junto con la lógica de JavaScript. La composición de piezas reutilizables facilita mantener elementos compartidos y evita repetir su implementación (React, s. f.-a). El proyecto declara React 19.1.0, React DOM 19.1.0, React Router DOM 7.6.2 y Vite 6.3.5. LoginPage y RegisterPage ensamblan componentes de react-app/src/components: AuthLayout, FormField, Alert, PrimaryButton, PasswordToggle, Brand, StoryPanel y ThemeToggle.

### Estado y eventos

El estado contiene datos que cambian durante el uso y afectan lo que la interfaz muestra. El Hook useState permite declarar estado local y actualizarlo en respuesta a eventos, como escribir en un campo o enviar un formulario (React, s. f.-b). En las páginas de acceso, el estado gestiona valores del formulario, mensajes de resultado, carga y visibilidad de la contraseña. React vuelve a representar la interfaz pertinente cuando esos valores cambian.

### Enrutamiento y herramientas

React Router relaciona las direcciones del navegador con componentes; BrowserRouter usa la History API y Routes/Route declaran las vistas (React Router, s. f.). BackToMe define rutas para login y registro. Vite ofrece un servidor de desarrollo con recarga rápida y un comando de compilación para generar los recursos de producción (Vite, s. f.).

## Aplicación de React en BackToMe

LoginPage controla correo, carnet y contraseña, valida los campos y envía POST a /api/auth/login. Si el servidor responde correctamente, el cliente guarda el token y el perfil en localStorage y abre dashboard.html. RegisterPage recoge nombre, apellido, carnet, carrera, correo y contraseña; valida los campos y consume POST a /api/auth/register. El servidor inicia el proceso de verificación del correo. Los estados de espera y error se presentan mediante componentes compartidos.

Tabla 1

*Componentes reutilizables del acceso React*

| Componente | Función en las pantallas de acceso |
|---|---|
| **AuthLayout** | Estructura compartida para login y registro. |
| **FormField** | Campo controlado con etiqueta, valor y eventos. |
| **Alert / PrimaryButton** | Mensaje de resultado y botón con estado de carga. |
| **PasswordToggle** | Mostrar u ocultar la contraseña. |
| **Brand / ThemeToggle / StoryPanel** | Identidad, tema visual y panel informativo. |

**Alcance de la migración.** El dashboard y los módulos de objetos, solicitudes y mensajes permanecen en HTML, CSS y JavaScript. React está aplicado de forma verificable en las

pantallas principales de acceso, pero la interfaz completa aún no está migrada. Esta delimitación identifica trabajo que corresponde continuar en la etapa final.

## Aportes de las integrantes

La integración combina migración del acceso y una API externa vinculada a los reportes. La Tabla 2 distingue las responsabilidades documentadas en el código y los materiales del proyecto.

Tabla 2

*Distribución de aportes observados en el proyecto*

| Integrante | Aporte en esta entrega |
|---|---|
| **Andrea** | Preparó las pantallas de inicio de sesión y registro en React, las rutas, los componentes reutilizables y el manejo de estado de formularios; conectó las vistas con los endpoints de autenticación. |
| **Kimberly (aporte propio)** | Integró Cloudinary al flujo de publicación: validación y carga firmada desde el servidor, almacenamiento de la URL segura en MySQL y nombres de variables en .env.example sin credenciales reales. |
| **Trabajo compartido** | Se conserva la API Express, MySQL/MariaDB, verificación de correo y módulos existentes de publicaciones, solicitudes y conversaciones. |

## API externa de Cloudinary

Cloudinary ofrece un Upload API que recibe imágenes por HTTPS y devuelve información del recurso, incluida una URL segura (Cloudinary, s. f.-b). En BackToMe, la integración enlaza ese servicio con la publicación para que la foto pertenezca al reporte y pueda mostrarse desde su URL.

Figura 1



*Flujo de datos de la fotografía en una publicación.*

El endpoint POST /api/objects requiere sesión y valida campos, formato (JPEG, PNG o WebP) y tamaño. Si Cloudinary está configurado, Express firma folder y timestamp con SHA-1 y envía un POST a /image/upload. La firma se genera en el servidor y el API Secret no se expone al cliente (Cloudinary, s. f.-a). La respuesta secure_url se guarda en foto_url junto con el reporte.

Sin credenciales, BackToMe conserva el almacenamiento local de fotos en MySQL. Cloudinary almacena imágenes, pero la cámara depende del selector de archivos y del dispositivo.

## Análisis técnico y seguridad

La API añade valor al flujo porque mantiene la imagen en un servicio externo y deja en la base una referencia para mostrarla. La firma se calcula en el backend con credenciales de entorno, sin enviar el API Secret a React. .env se excluye del repositorio y .env.example contiene nombres de variables sin claves. La migración de contenido amplía foto_url para el mecanismo local alternativo.

La validación de tipo, tamaño y sesión evita algunas entradas no previstas y vincula el reporte con una cuenta autenticada. Antes de un despliegue público se deberían revisar límites de dimensiones y bytes, restricciones de Cloudinary, CORS y medidas contra abuso. Cloudinary no reemplaza las reglas de autorización de BackToMe. La API tampoco añade por sí sola una función de cámara.

En relación con la rúbrica, la aplicación demuestra React en dos pantallas esenciales, uso de estado y componentes reutilizables, además de datos externos integrados al módulo de objetos. La migración parcial de los demás módulos debe ampliarse en la entrega final. La documentación visual y una demostración práctica complementan este análisis técnico.

## Conclusiones

React organiza las pantallas de acceso de BackToMe mediante componentes, estado y rutas. Su apariencia puede mantenerse similar a la versión anterior porque se reutilizan estilos; la mejora principal está en la estructura y reutilización del código, no en un cambio visual automático. Vite apoya el desarrollo y la compilación.

Cloudinary conecta los reportes con un servicio externo: el servidor valida y firma la foto, recibe su URL segura y la guarda en MySQL. Andrea desarrolló el acceso en React y Kimberly integró Cloudinary. En la etapa final conviene ampliar React a más módulos.

## Referencias

Cloudinary. (s. f.-a). *Generating authentication signatures*. Recuperado el 9 de noviembre de 2026, de https://cloudinary.com/documentation/authentication_signatures

Cloudinary. (s. f.-b). *Upload API reference*. Recuperado el 9 de noviembre de 2026, de https://cloudinary.com/documentation/image_upload_api_reference

React. (s. f.-a). *Describing the UI*. Recuperado el 9 de noviembre de 2026, de https://react.dev/learn/describing-the-ui

React. (s. f.-b). *Managing state*. Recuperado el 9 de noviembre de 2026, de https://react.dev/learn/managing-state

React Router. (s. f.). *Routing*. Recuperado el 9 de noviembre de 2026, de https://reactrouter.com/start/declarative/routing

Vite. (s. f.). *Getting started*. Recuperado el 9 de noviembre de 2026, de https://vite.dev/guide/